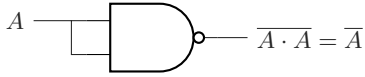


Related Questions

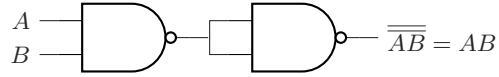
Q: Describe universality of NAND Gate.

We can design any gate using only NAND gate, therefore NAND gate is an universal gate.

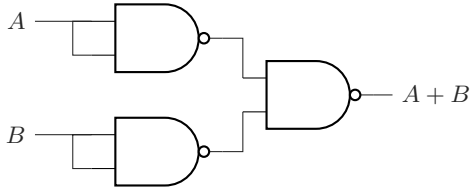
NOT gate:



AND gate:

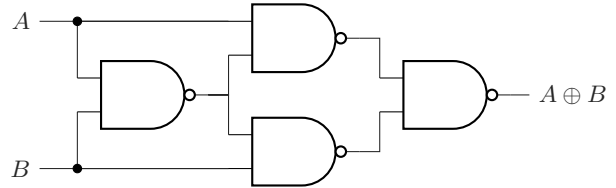


OR gate:



$$\bar{\bar{A}}\bar{\bar{B}} = \bar{A} + \bar{B} = A + B$$

XOR gate:

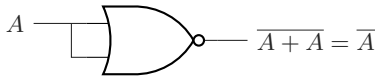


$$\begin{aligned} \bar{\bar{A}}\bar{\bar{B}} \cdot \bar{\bar{B}}\bar{\bar{A}} &= A\bar{B} + B\bar{A} \\ &= A(\bar{A} + \bar{B}) + B(\bar{A} + \bar{B}) \\ &= (\bar{A} + \bar{B})(A + B) = A \oplus B \end{aligned}$$

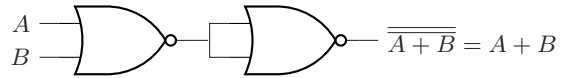
Q: Describe universality of NOR Gate.

We can design any gate using only NOR gate, therefore NOR gate is an universal gate.

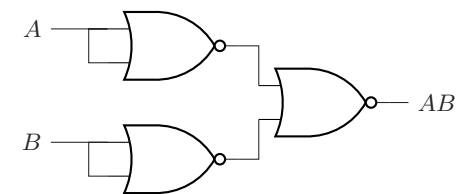
NOT gate:



OR gate:

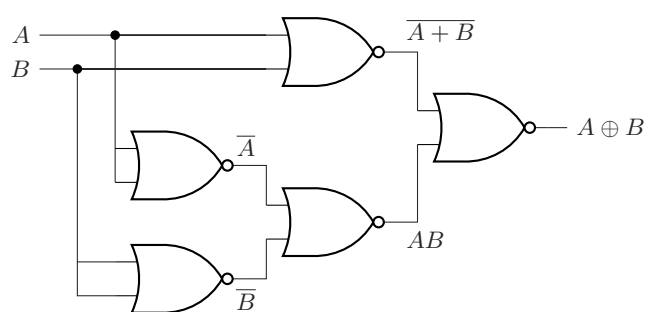


AND gate:



$$\bar{\bar{A}} + \bar{\bar{B}} = \bar{\bar{A}}\bar{\bar{B}} = AB$$

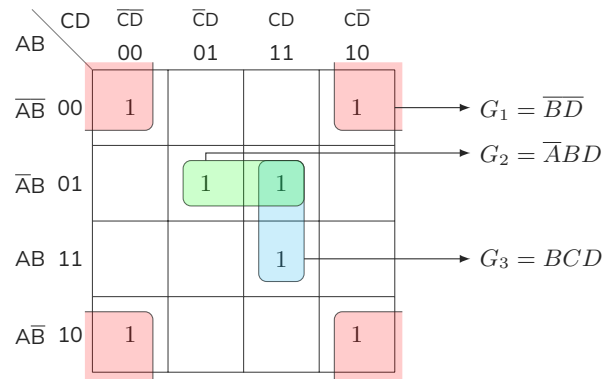
XOR gate:



$$\overline{(A + B) + (AB)} = \overline{(A + B)} \cdot \overline{(AB)} = A \oplus B$$

Logic Simplification

Q: Implement the given function using K-map $F(A, B, C, D) = \sum \{0, 2, 5, 7, 8, 10, 15\}$



$$\therefore Y = \overline{B}D + \overline{A}BD + BCD$$

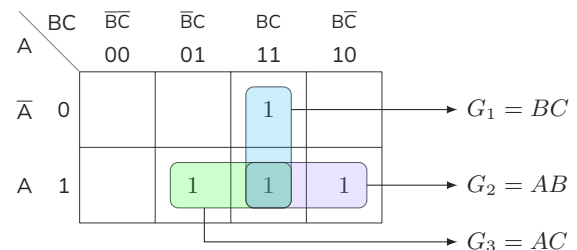
Q: Design a logic circuit from the given statement:

Suppose A,B,C are inputs and Y is the output. Output will be high when majority of the inputs are high.

	A	B	C	Y
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

$\rightarrow \overline{A}BC$
 $\rightarrow A\overline{B}C$
 $\rightarrow AB\overline{C}$
 $\rightarrow ABC$

$$Y = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$



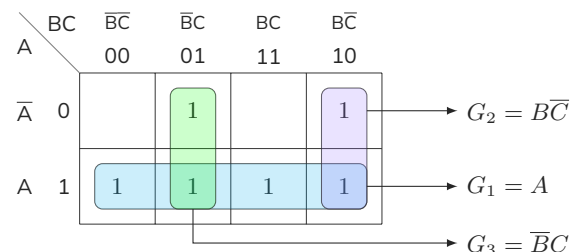
$$Y = AB + BC + AC$$

Q: Design a logic circuit that follows the following requirements:

- (i) Output X will be equal to A when B and C are same.
- (ii) Output X will remain high when B and C are different.

	A	B	C	X	
0	0	0	0	0	
1	0	0	1	1	$\rightarrow \overline{A}BC$
2	0	1	0	1	$\rightarrow \overline{A}B\overline{C}$
3	0	1	1	0	
4	1	0	0	1	$\rightarrow A\overline{B}\overline{C}$
5	1	0	1	1	$\rightarrow A\overline{B}C$
6	1	1	0	1	$\rightarrow AB\overline{C}$
7	1	1	1	1	$\rightarrow ABC$

$$X = \overline{A}BC + \overline{A}B\overline{C} + A\overline{B}\overline{C} + A\overline{B}C + AB\overline{C} + ABC$$



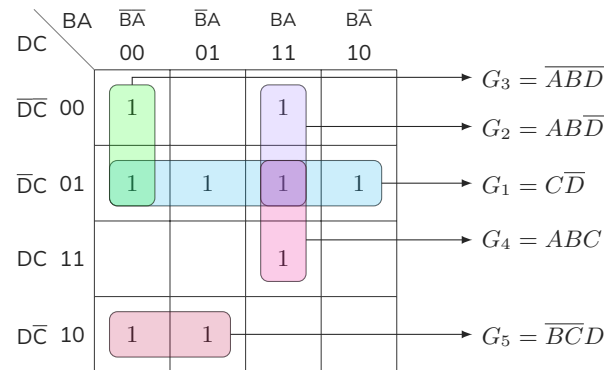
$$\begin{aligned}
 X &= A + \overline{B}C + \overline{B}C \\
 &= A + (B \oplus C)
 \end{aligned}$$

Q: Design a logic circuit that follows the following requirements:

A 4-bit binary number is represented as D,C,B,A where D is MSB and A is LSB. Design a logic circuit that will produce a high output whenever the binary number is greater than 0010 and less than 1010 or all inputs are low or all inputs are high.

Solution:

	D	C	B	A	Y	
0	0	0	0	0	1	$\rightarrow \overline{ABCD}$
1	0	0	0	1	0	
2	0	0	1	0	0	
3	0	0	1	1	1	$\rightarrow \overline{ABC}\overline{D}$
4	0	1	0	0	1	$\rightarrow \overline{A}\overline{B}\overline{C}\overline{D}$
5	0	1	0	1	1	$\rightarrow \overline{A}\overline{B}\overline{C}D$
6	0	1	1	0	1	$\rightarrow \overline{A}\overline{B}CD$
7	0	1	1	1	1	$\rightarrow \overline{A}BCD$
8	1	0	0	0	1	$\rightarrow \overline{A}\overline{B}\overline{C}D$
9	1	0	0	1	1	$\rightarrow \overline{A}\overline{B}CD$
10	1	0	1	0	0	
11	1	0	1	1	0	
12	1	1	0	0	0	
13	1	1	0	1	0	
14	1	1	1	0	0	
15	1	1	1	1	1	$\rightarrow ABCD$



$$X = \overline{A}\overline{B}\overline{D} + AB\overline{D} + \overline{B}\overline{C}\overline{D} + ABC + C\overline{D}$$

Q: Design a logic circuit that follows the following requirements:

Suppose A,B,C,D are inputs and Y is the output. Output will be high when A=B and C≠D.

Solution:

	A	B	C	D	Y	
0	0	0	0	0	0	
1	0	0	0	1	1	$\rightarrow \overline{A}\overline{B}\overline{C}D$
2	0	0	1	0	1	$\rightarrow \overline{A}\overline{B}C\overline{D}$
3	0	0	1	1	0	
4	0	1	0	0	0	
5	0	1	0	1	0	
6	0	1	1	0	0	
7	0	1	1	1	0	
8	1	0	0	0	0	
9	1	0	0	1	0	
10	1	0	1	0	0	
11	1	0	1	1	0	
12	1	1	0	0	0	
13	1	1	0	1	1	$\rightarrow A\overline{B}\overline{C}D$
14	1	1	1	0	1	$\rightarrow A\overline{B}C\overline{D}$
15	1	1	1	1	0	

$$Y = \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + A\overline{B}\overline{C}D + A\overline{B}C\overline{D}$$

AB \ CD	$\overline{C}\overline{D}$ 00	$\overline{C}D$ 01	$C\overline{D}$ 11	CD 10
$\overline{A}\overline{B}$ 00		1		1
$\overline{A}B$ 01				
$A\overline{B}$ 11		1		1
AB 10				

$$Y = \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + A\overline{B}\overline{C}D + A\overline{B}C\overline{D}$$

Digital Arithmetic

$$\begin{array}{r} 0 \\ + 0 \\ \hline 0 \\ \text{Carry} \quad \text{Sum} \end{array}$$

$$\begin{array}{r} 1 \\ + 0 \\ \hline 1 \\ \text{Carry} \quad \text{Sum} \end{array}$$

$$\begin{array}{r} 0 \\ + 1 \\ \hline 1 \\ \text{Carry} \quad \text{Sum} \end{array}$$

$$\begin{array}{r} 1 \\ + 1 \\ \hline 0 \\ 1 \\ \text{Carry} \quad \text{Sum} \end{array}$$

Q: What is a Combinational circuit?

A combinational circuit is one in which we combine the various different gates to form a circuit. The output of the combinational circuit is determined by the present inputs, regardless of the previous inputs.

Q: What is a Sequential circuit?

A sequential circuit is one in which previous state output is used as the input to the next component. It also has memory elements. There are two types of sequential circuits

- (a) Synchronous
- (b) Asynchronous

Q: Show the differences between Combinational circuit and Sequential circuit.

Combinational circuit	Sequential circuit
In Combinational Circuit, output depends on the present state only.	In Sequential Circuit, output depends on present state as well as the past state.
Does not require any memory element to store the output.	Requires a memory element to store the previous state output.
Clock signal is required.	Clock signal isn't required.
Easy to design.	Difficult to design.

Half Adder

Half Adder is a combinational logic circuit which is designed by connecting one **XOR** gate and one **AND** gate. The adder circuit as two inputs *A* and *B* which adds two binary numbers and generates a *Carry* and *Sum*.

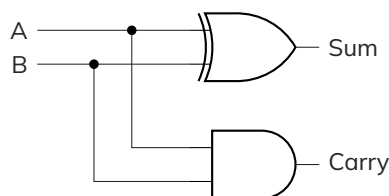
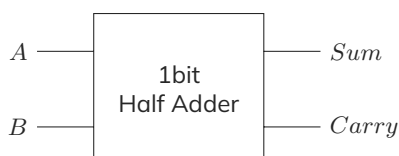


Figure 1: Half Adder



Logical Expression:

$$\text{Sum} = \overline{A}B + A\overline{B} = A \oplus B$$

$$\text{Carry} = A.B$$

Truth Table:

<i>A</i>	<i>B</i>	<i>Sum</i>	<i>Carry</i>
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Full Adder

Full Adder is combinational logic circuit that is consist of 2 XOR gate, 2 AND gate and 1 OR gate. Full Adder performs an addition operation on three one-bit binary numbers A , B , C_{in} and produces a sum of the three inputs and carry value designated as C_{out} .

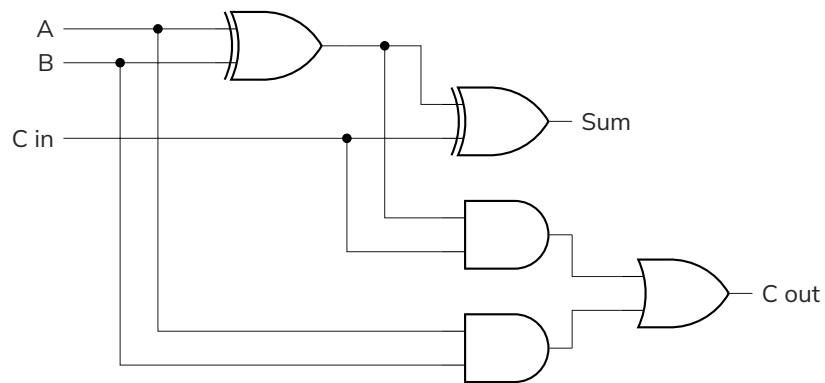
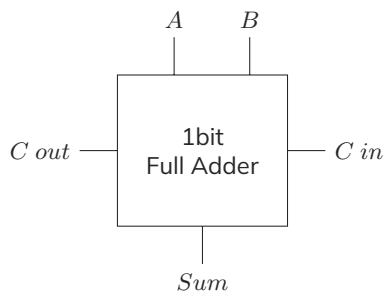


Figure 2: Full Adder



Logical Expression:

$$\begin{aligned} \text{sum} &= \overline{A}\overline{B}C_{in} + \overline{A}B\overline{C}_{in} + A\overline{B}\overline{C}_{in} + ABC_{in} \\ &= \overline{A}(\overline{B}C_{in} + B\overline{C}_{in}) + A(\overline{B}\overline{C}_{in} + BC_{in}) \\ &= \overline{A}(B \oplus C_{in}) + A(\overline{B \oplus C_{in}}) \\ &= A \oplus B \oplus C_{in} \end{aligned}$$

$$\begin{aligned}
 \text{carry} &= \overline{A}BC_{in} + A\overline{B}C_{in} + AB\overline{C_{in}} + ABC_{in} \\
 &= C_{in}(\overline{A}B + A\overline{B}) + AB(\overline{C_{in}} + C_{in}) \\
 &= C_{in}(A \oplus B) + AB
 \end{aligned}$$

Truth Table:

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

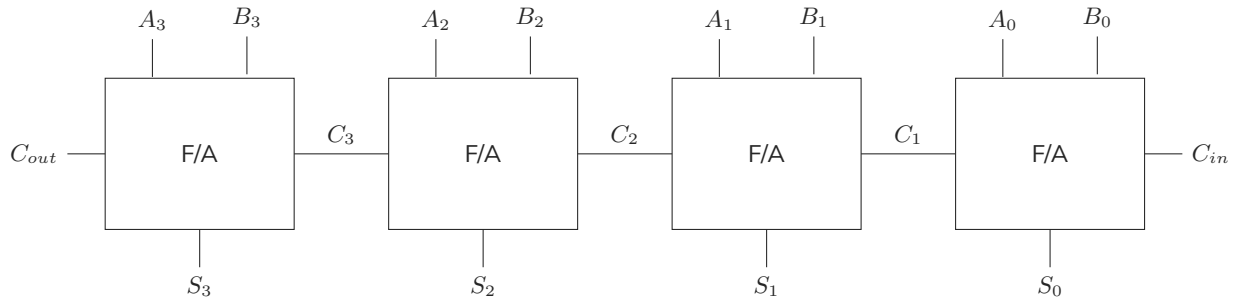


Figure 3: 4 Bit Parallel Adder

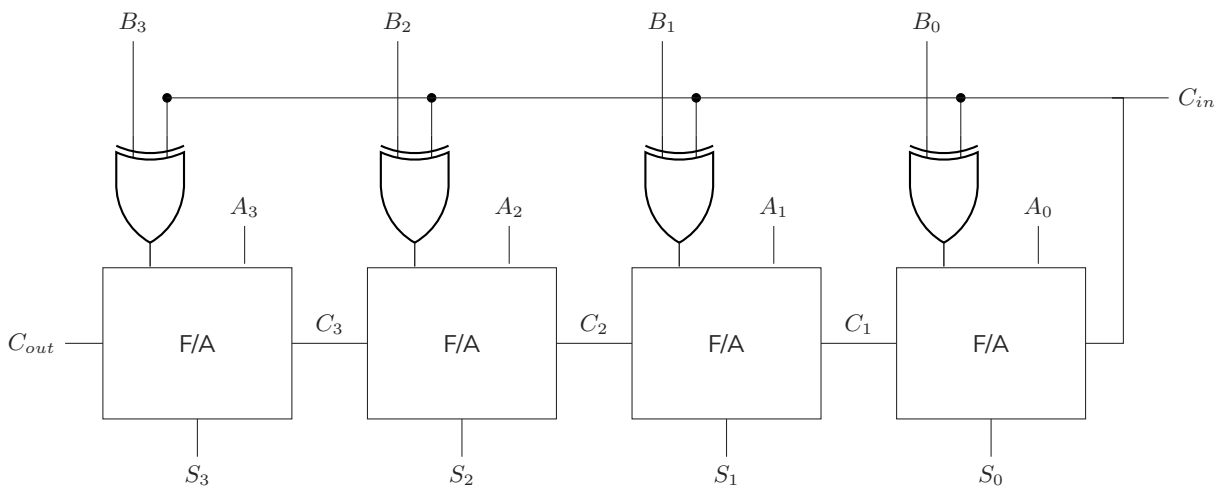


Figure 4: 4-bit Parallel Adder-Subtractor

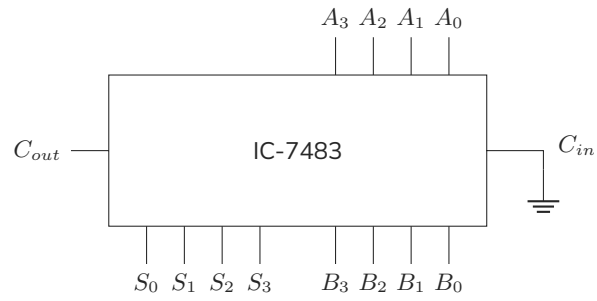


Figure 5: Adder using IC-7483

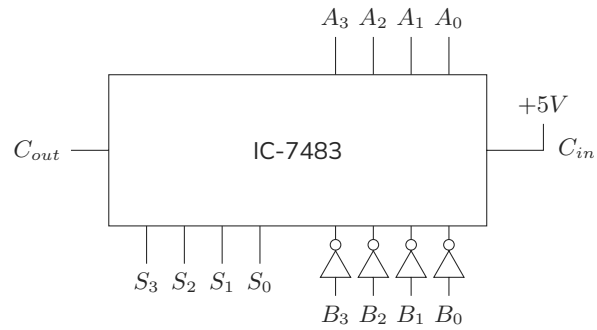


Figure 6: Subtractor using IC-7483

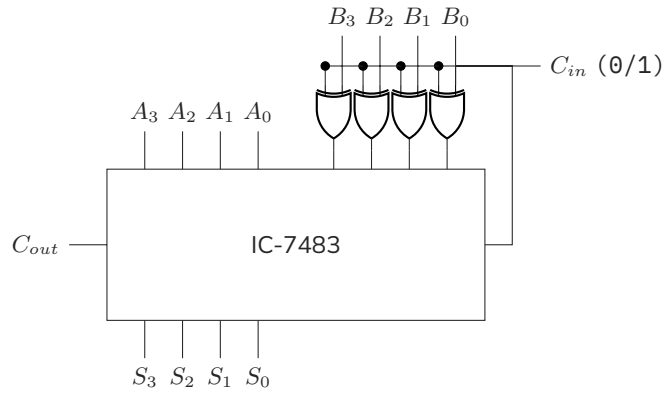


Figure 7: Adder-Subtractor using IC-7483

$$A \oplus 0 = A$$

$$A \oplus 1 = \overline{A}$$

$$A - B = A + (-B) = A + (\overline{B} + 1)$$

Flip-Flop and Related Devices

Q: What is Flip-Flop?

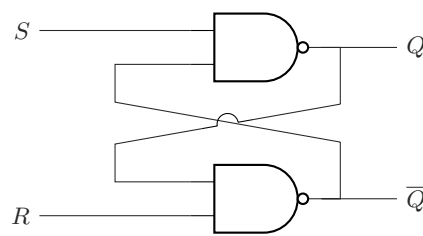
A flip-flop is a circuit with two stable states that can be used to store binary data. The stored data can be changed by applying varying inputs. This is the smallest and fundamental building blocks of the memory unit used in computer, RAM and other memory devices. This device has many inputs and exactly two outputs. One is opposite to the other.

Q: Why do we use Flip-Flop?

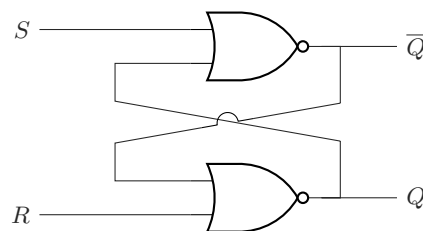
A flip flop serves as a fundamental building blocks of the memory unit memory unit that is capable of retaining a single bit of data, either a '0' or a '1'.

S-R Flip-Flop

The SET-RESET (SR) flip-flop can be designed with the help of two NAND gate or NOR gate. This flip-flop is also called *S-R Latch*.



S	R	Q
0	0	Invalid
0	1	1
1	0	0
1	1	No Change



S	R	Q
0	0	No Change
0	1	0
1	0	1
1	1	Invalid

Figure 8: Circuit Diagram of *S-R* Flip-Flop and Truth Table

State diagram:

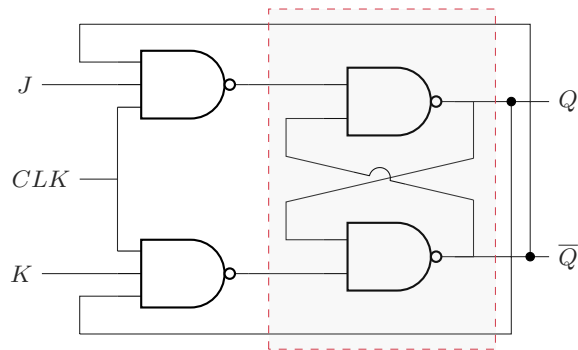
Here

Operation:

Here

J-K Flip-Flop

It is an enhanced version of the S - R (Set-Reset) flip-flop, addressing the invalid state associated with the S - R flip-flop. It has two inputs J and K , along with the clock signal.



J	K	CLK	Q
0	0	$\uparrow$	No Change
0	1	$\uparrow$	0
1	0	$\uparrow$	1
1	1	$\uparrow$	Toggle

Figure 9: Circuit Diagram of J - K Flip-Flop and Truth Table

State diagram:

Here

Operation:

When $J = 0$ and $K = 0$

No change in the output

When $J = 0$ and $K = 1$

$$\left. \begin{array}{l} \text{If initially } Q = 0, \text{ finally } Q = 0 \\ \text{If initially } Q = 1, \text{ finally } Q = 0 \end{array} \right\} \text{Reset condition}$$

When $J = 1$ and $K = 0$

$$\left. \begin{array}{l} \text{If initially } Q = 0, \text{ finally } Q = 1 \\ \text{If initially } Q = 1, \text{ finally } Q = 1 \end{array} \right\} \text{Set condition}$$

When $J = 1$ and $K = 1$

$$\left. \begin{array}{l} \text{If initially } Q = 0, \text{ finally } Q = 1 \\ \text{If initially } Q = 1, \text{ finally } Q = 0 \end{array} \right\} \text{Toggle condition}$$

D Flip-Flop

D flip-flop is a single input version of S - R flip-flop. It has one input along with the clock signal.

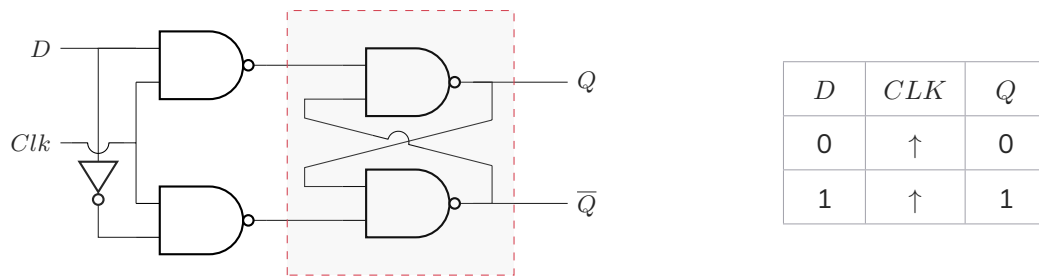


Figure 10: Circuit Diagram of D Flip-Flop and Truth Table

State diagram:

Here

Operation:

Here

T Flip-Flop

T flip-flop is a single input version of S - R flip-flop. It has one input along with the clock signal.

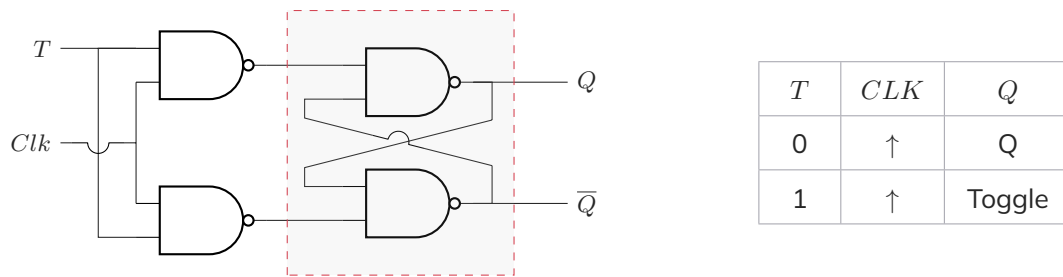


Figure 11: Circuit Diagram of D Flip-Flop and Truth Table

State diagram:

Here

Operation:

Here

4-bit BCD Adder

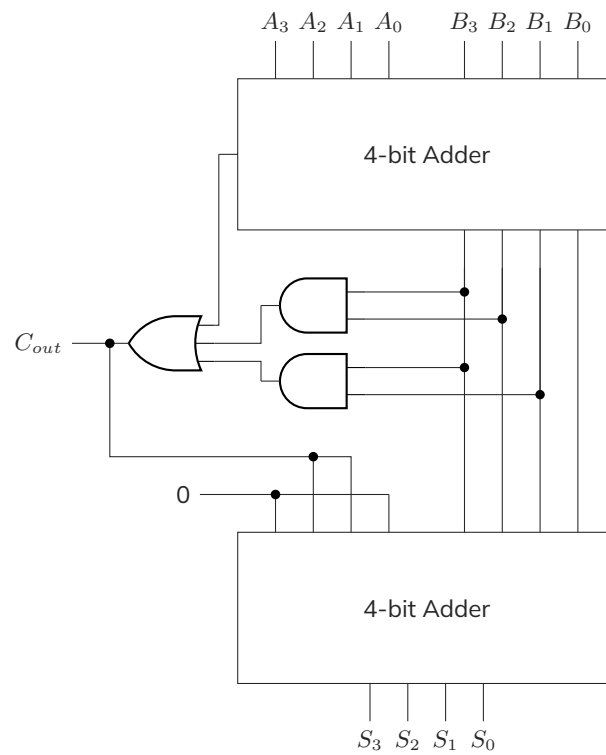
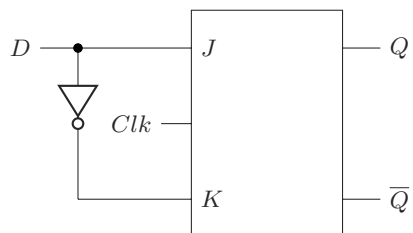
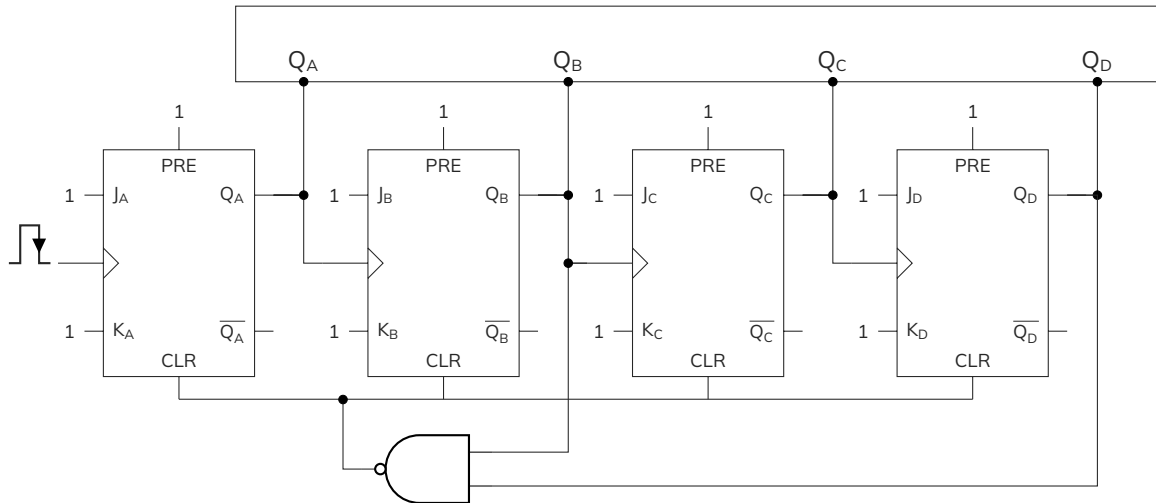


Figure 12: Block Diagram of a BCD Adder

Convert J - K Flip-Flop to D Flip-Flop



MOD 10 Up Counter



	D	C	B	A	
0	0	0	0	0	PRE=1
1	0	0	0	1	CLR=1
2	0	0	1	0	
3	0	0	1	1	
4	0	1	0	0	
5	0	1	0	1	
6	0	1	1	0	
7	0	1	1	1	
8	1	0	0	0	
9	1	0	0	1	
10	1	0	1	0	PRE=1
Reset Condition = 1 0 1 0					CLR=0

Operation

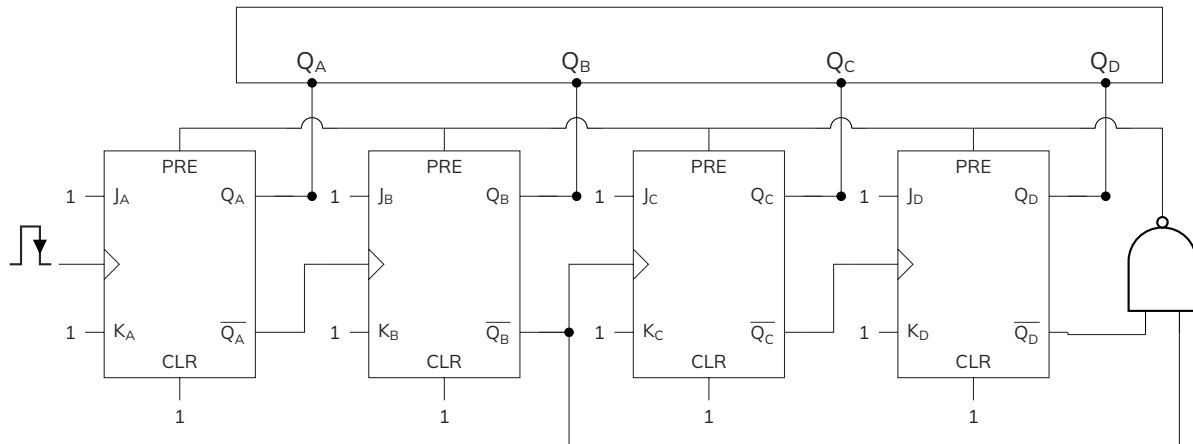
From (0 to 9) $\Rightarrow$ PRE=1 CLR=1
Normal Clock Operation

For (10) $\Rightarrow$ PRE=1 CLR=0
 $A = B = C = D = 0$
So it counts 0000

$\therefore$ It counts from 0 to 9

$\therefore$ This is MOD 10 up counter

MOD 10 Down Counter



	D	C	B	A	
15	1	1	1	1	PRE=1
14	1	1	1	0	CLR=1
13	1	1	0	1	
12	1	1	0	0	
11	1	0	1	1	
10	1	0	1	0	
9	1	0	0	1	
8	1	0	0	0	
7	0	1	1	1	
6	0	1	1	0	
5	0	1	0	1	PRE=0
Reset Condition = 0 1 0 1					CLR=1

Operation

From (15 to 6) $\Rightarrow$ PRE=1 CLR=1
Normal Clock Operation

For (5) $\Rightarrow$ PRE=0 CLR=1
 $A = B = C = D = 1$
 So it counts 1111

∴ It counts from 15 to 6

∴ This is MOD 10 down counter

Synchronous Counter

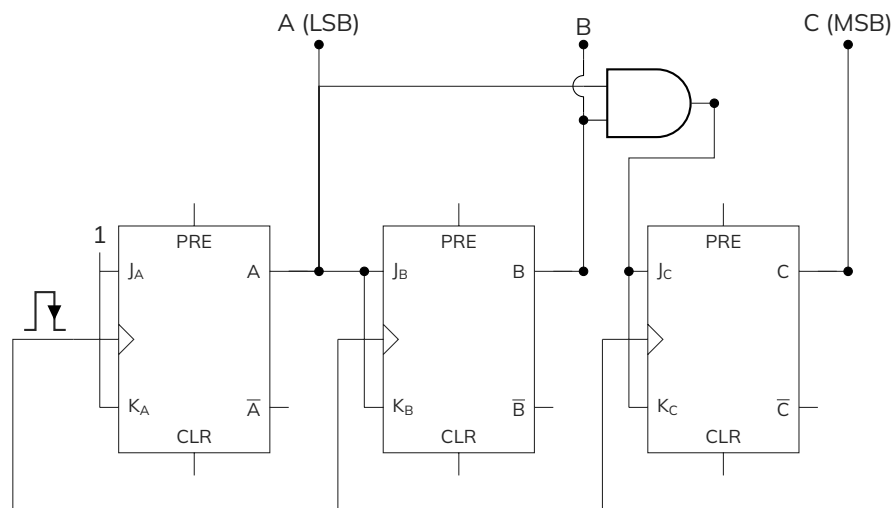
Q: Define Asynchronous counter?

The counter where all the outputs do not response together, that counter is called asynchronous counter or ripple counter.

Q: Define Synchronous counter?

The counter where all the outputs response together, that counter is called asynchronous counter or parallel counter.

3-bit Synchronous Up Counter (MOD 8)



	C	B	A
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
0	0	0	0

Operation

Initially $A = B = C = 0$

A : is always toggle, $J_A = K_A = 1$

B : if $A = 0$ then, $J_B = K_B = 0$

: if $A = 1$ then, $J_B = K_B = 1$

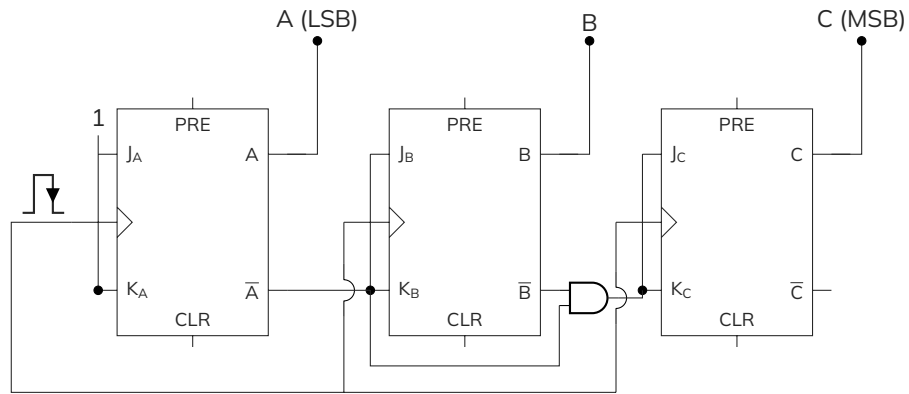
$\therefore J_B = K_B = A$

C : if $A=B=1$ then, $J_C = K_C = 1$

otherwise, $J_C = K_C = 0$

$\therefore J_C = K_C = A.B$

3-bit Synchronous Down Counter (MOD 8)



	C	B	A
7	1	1	1
6	1	1	0
5	1	0	1
4	1	0	0
3	0	1	1
2	0	1	0
1	0	0	1
0	0	0	0
7	1	1	1

Operation

Initially $A = B = C = 1$

A : is always toggle, $J_A = K_A = 1$

B : if $A = 0$ then, $J_B = K_B = 1$

: if $A = 1$ then, $J_B = K_B = 0$

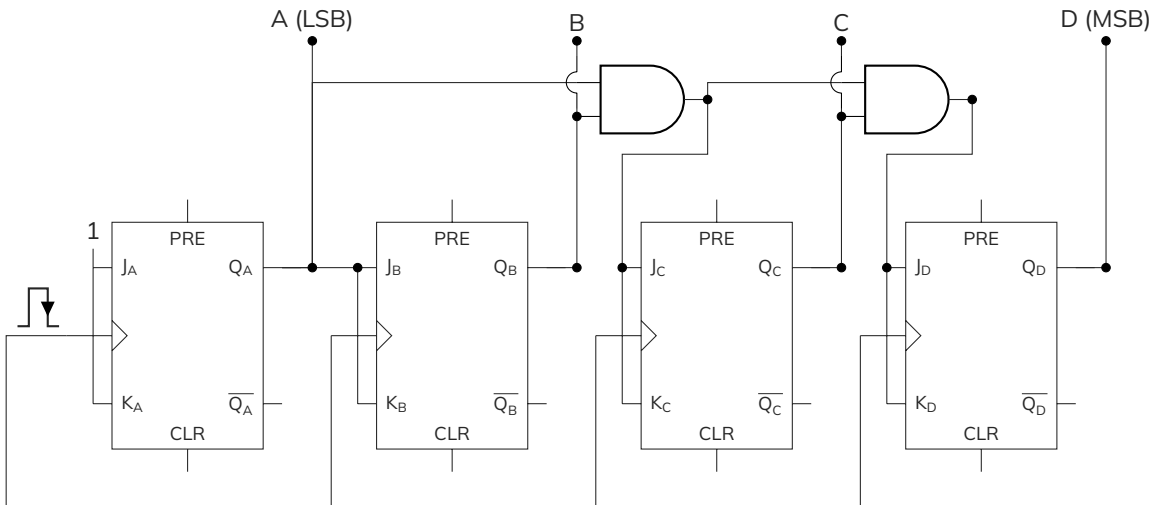
$\therefore J_B = K_B = \bar{A}$

C : if $A=B=0$ then, $J_C = K_C = 1$

otherwise, $J_C = K_C = 0$

$\therefore J_C = K_C = \bar{A}.\bar{B}$

4-bit Synchronous Up Counter (MOD 16)



	D	C	B	A
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

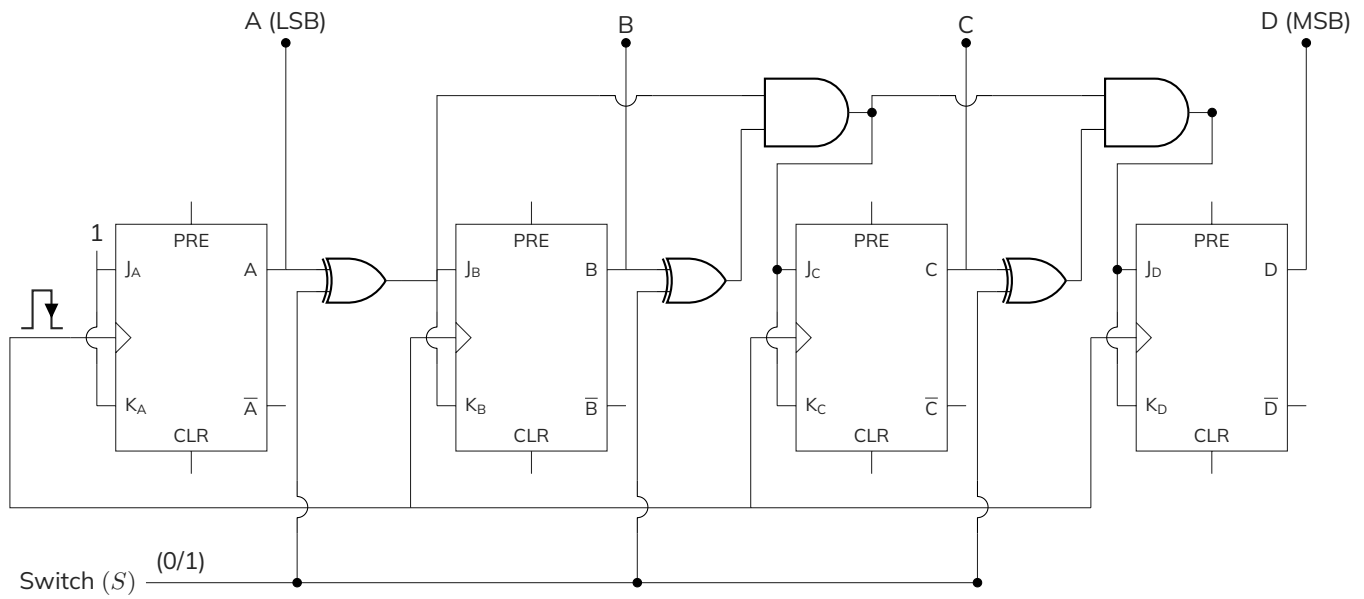
Operation

Initially $A = B = C = D = 0$

A : is always toggle, $J_A = K_A = 1$

$$\begin{array}{ll} B & : \text{ if } A = 0 \text{ then,} & J_B = K_B = 0 \\ & : \text{ if } A = 1 \text{ then,} & J_B = K_B = 1 \\ \therefore & J_B = K_B = A \end{array}$$
$$\begin{aligned} C &: \text{ if } A=B=1 \text{ then,} & J_C = K_C = 1 \\ & \text{ otherwise,} & J_C = K_C = 0 \\ \therefore J_C = K_C = A.B \end{aligned}$$
$$\begin{aligned} D &: \text{ if } A=B=C=1 \text{ then,} & J_D = K_D = 1 \\ & \text{ otherwise,} & J_D = K_D = 0 \\ \therefore J_D = K_D = A.B.C \end{aligned}$$

4-bit Synchronous Up/Down Counter (MOD 16)



	D	C	B	A
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

Operation

When $S = 0$, then $[A \oplus 0 = A]$

$$J_A = K_A = 1$$

$$J_B = K_B = A$$

$$J_C = K_C = A.B$$

$$J_D = K_D = A.B.C$$

It is up counter

When $S = 1$, then $[A \oplus 1 = \overline{A}]$

$$J_A = K_A = 1$$

$$J_B = K_B = \overline{A}$$

$$J_C = K_C = \overline{A}.\overline{B}$$

$$J_D = K_D = \overline{A}.\overline{B}.\overline{C}$$

- It is down counter

Propagation Delay

For any device, to give the input, and response for output, it takes some time, this time is called propagation Delay.

$$J.K \text{ FF} \Rightarrow t_{pd} = 50ns = 50 \times 10^{-9}s$$

$$\text{AND gate} \Rightarrow t_{pd} = 20ns = 20 \times 10^{-9}s$$

For asynchronous counter: Total $t_{pd} = t_{pd} \text{ for one } J.K \text{ FF} \times \text{Number of FF}$

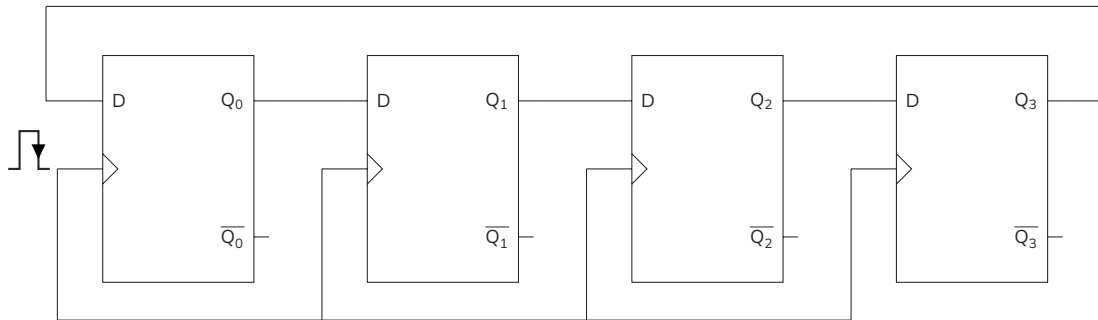
For synchronous counter: Total $t_{pd} = t_{pd} \text{ for one } J.K \text{ FF} + t_{pd} \text{ for one AND gate}$

Shift Counter

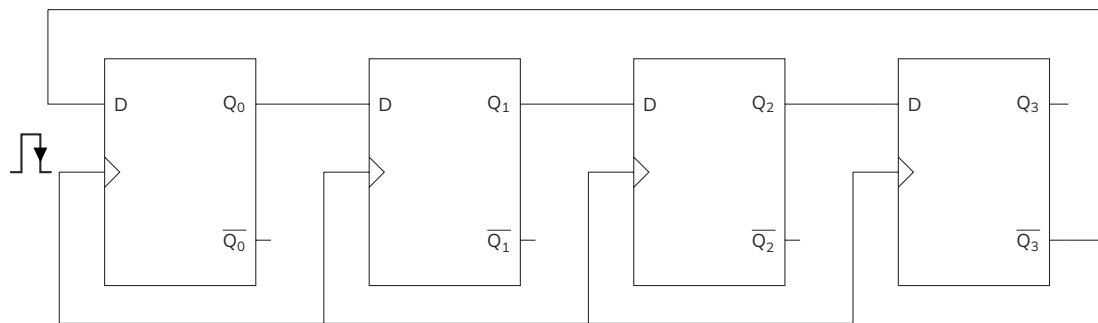
(1) Ring Counter [MOD Number = Number of FF]

(2) Johnson Counter [MOD Number = $2 \times \text{Number of FF}$]

MOD 4 Ring Counter



MOD 8 Johnson Counter



Counter Summery

Q: For each of the following statements, indicates the types of counter being described:

Answers

- | | |
|--|--|
| 1. Each FF is clocked at the same time. | Synchronous counter / Parallel counter |
| 2. Each FF divides the frequency at its clock pulse by 2. | Any counter |
| 3. The counter sequence is
111 → 110 → 101 → 100 → 011 → 010 → 001 → 000. | MOD 8 Down counter |
| 4. The counter has 10 distinct states. | MOD 10 counter |
| 5. The total delay is the sum of the individual FF's delay. | Asynchronous counter / Ripple counter |
| 6. The counter can count in either direction. | Up/Down counter |
| 7. The counter counts from 0 to 9. | MOD 10 Up counter |
| 8. The MOD number is always twice the number of FFs. | Johnson counter |
| 9. The total delay is the sum of one FF's delay and one AND gate's delay. | Synchronous counter |
| 10. The MOD number is always equal to the number of FFs. | Ring counter |

Counter Formulas

MOD Number:

Asynchronous counter / Ripple counter = 2^N

Synchronous counter / Parallel counter = 2^N

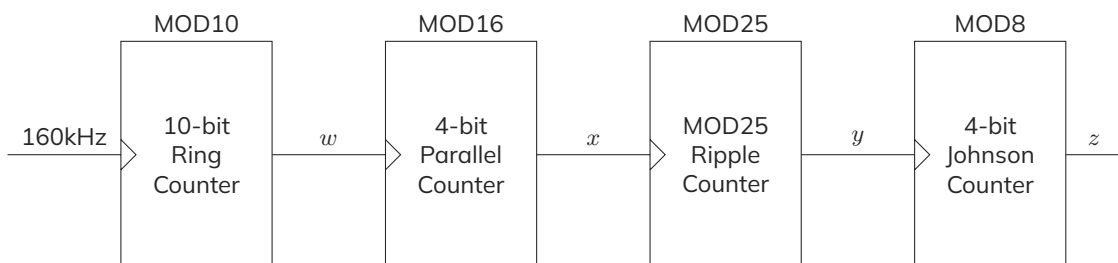
Ring counter = N

Johnson counter = $2N$

Here, N = Number of FF

$$\text{Output Frequency} = \frac{\text{Input Frequency}}{\text{MOD Number}}$$

Q: Find the values of w , x , y , z



$$w = \frac{160\text{kHz}}{10} = 16\text{kHz}$$

$$x = \frac{16\text{kHz}}{16} = 1000\text{Hz}$$

$$y = \frac{1000\text{Hz}}{25} = 40\text{Hz}$$

$$z = \frac{40\text{Hz}}{8} = 5\text{Hz}$$